

Analyst Intern — Take-Home Assignment

Welcome, and thanks for your interest in the role.

This exercise is built from the kind of work the team actually does: messy player telemetry, a product stakeholder who wants an answer, and a decision waiting on it. There is no single correct submission. We are far more interested in how you think than in whether you produced a specific number.

Time budget: roughly **8–10 hours**, and you have **5 calendar days** to submit. We do not expect it in one sitting.

We would genuinely rather have three parts done well than five done thinly. **If you run short, prioritise in this order: 1, 2, 5, then 3, then 4** — and tell us what you dropped and what you would have done with more time. That is a legitimate and well-regarded answer, and how you triage is itself something we are looking at.

The situation

You have joined the Analytics team supporting a live-service mobile game, roughly two years past launch. It is well out of its launch spike and into the long tail: the player base is stable but no longer growing on its own, and most new players now arrive through paid marketing rather than finding the game themselves.

The studio's goal this year is to grow revenue without growing marketing spend. That is the frame for everything below. Quarterly planning starts in three weeks, and the budget conversation is the one your work feeds into.

How the game makes money

Players download it free. A small minority — a few percent — ever spend real money, buying packs of premium currency. Those players carry the revenue.

Players also opt in to watch **rewarded video ads** in exchange for in-game currency, with per-session and per-day caps on how many they can watch. This does earn money, but not in a way you can see here: the ad network bills the studio monthly and that sits in a finance system, **not in player telemetry**.

So there is no ad revenue in this dataset. `ad_view` tells you about ad *engagement*, and `reward_amount` is the in-game currency the player received — a cost to the game economy, not money earned. Wherever this brief says "revenue," it means in-app purchases.

Beyond that, players progress through levels and spend three in-game currencies: premium (bought or granted), social, and a soft "grind" currency earned through play.

What is not in this extract

This is **player telemetry only** — no marketing cost data (no cost per install or spend by channel), no ad revenue, no support tickets or survey responses.

That is a deliberate constraint. If answering a question properly needs a number you have not been given, **say so, and say what you would do with it** — naming a missing input beats quietly working around it. Where a decision depends on an unknown, reason with it as one: "*this is worth doing if a retained player is worth more than \$X*" is useful; "*we cannot know*" is not.

The data

Five CSV files in `csv/` cover **~80,000 players across 120 days**, 2026-01-01 to 2026-04-30 — about 667,000 active player-days. It is anonymised production telemetry, and imperfect in the ways production data is imperfect; part of your job is to figure out how.

Load them into whatever environment you work in. Nothing here needs a database server, and we have no preference about how you set yourself up.

Worth checking early, because it affects more than one deliverable: **install volume is not flat across this window**. Look at its shape before pooling players from different weeks.

There is deliberately **no** exhaustive data dictionary — the columns and grains below are what a new joiner would actually get.

`player_profile` — one row per player

Column	Notes
<code>device_id</code>	Player identifier
<code>original_device_id</code>	Populated when this device has been linked to an earlier one
<code>install_date</code>	
<code>platform</code> , <code>country</code> , <code>acquisition_channel</code>	
<code>country_tier</code>	1–4. Standard internal grouping of countries by revenue potential; Tier 1 is the highest
<code>first_purchase_date</code>	Null if the player has never purchased

player_day — one row per player per active day

Column	Notes
device_id , activity_date	activity_date is stored as an integer, YYYYMMDD
player_level	Level at end of day
session_count , playtime_minutes	
app_version	

purchase — one row per real-money transaction

Column	Notes
purchase_id , device_id , event_ts	
pack_name , usd_amount , currency_granted	
sale_flag	Whether a promotion was active

currency_spend — one row per in-game currency spend event

Column	Notes
device_id , event_ts	
spend_type	paid / free / subscription — where the spent currency originated
currency_type	PREMIUM / SOCIAL / GRIND
currency_value , spend_category	
player_level , premium_balance , social_balance , grind_balance	Values at time of event

ad_view — one row per rewarded-ad interaction

Column	Notes
device_id , event_ts	
status	started / completed / abandoned
placement , reward_type , reward_amount	
ad_daily_count , ad_daily_cap	The player's ad count and cap at that moment

What to deliver

1. Get to know the data, in SQL (*aim for 2–2.5 hours*)

Four sets of numbers the team refers to constantly. **Include your SQL for each** — we read the queries, not just the outputs, and we will check the results against our own.

Two general warnings before you start. First, **be careful about what belongs in the numerator and the denominator**, and about which rows should be counted at all. Second, several of these are wrong in a way that still looks plausible, so sanity-check totals against each other rather than trusting one query.

(a) Daily health. By day, split by platform:

- Daily active players
- Gross revenue
- ARPDAU (average revenue per daily active player)
- Share of active players who made a purchase that day

(b) A retention cohort table. Group players by **install week**, and show the share of each cohort still active on day 1, day 7, day 14 and day 30 of their life.

Present it as a grid — cohorts down, day-milestones across. Think carefully about what belongs in the cells for recent cohorts, and say in one line what rule you applied and why.

(c) Spender behaviour. Three numbers:

- **Revenue concentration.** What share of total revenue comes from the top 1%, top 10% and top 50% of paying players? State which population you ranked — payers only, or everyone — because the two give very different answers and both are defensible.
- **Time to first purchase.** For players who ever pay, how long after installing do they pay for the first time? A distribution is more useful than an average. Include the share who never pay.
- **Repeat purchase gap.** Among players who buy more than once, how long between consecutive purchases? A window function will be much less painful here than a self-join.

(d) Rewarded ad engagement. Players have a per-day cap on rewarded ads (see `ad_daily_count` and `ad_daily_cap`).

- What share of ad-watching player-days hit the cap? Break this out by cap value — not everyone has the same one.
- Plot or tabulate total daily ad views across the window, and comment on anything that looks off. If you find something, say what you would do about it rather than just noting it.

If players are routinely hitting the cap, that is a product question as much as a data one — say what you would tell the team.

Then, a short note — a few bullets is fine — on any data quality issues you hit across all four parts and how you handled each. This note is worth real marks. We would rather see three issues found and handled thoughtfully than seven listed mechanically.

2. One deep-dive analysis, with a recommendation

Pick **one** of the two questions below and answer it properly. Both are real questions that landed on the team this quarter.

(a) "ARPDau is falling. Fix it."

Comparing the first 30 days of the window to the last 30, overall ARPDau is down roughly **16%**. Leadership wants to know what broke.

When you split the same period by `country_tier`, though, every single tier is flat or improving.

(A note on method, freely given: individual countries carry too few transactions for their ARPDau trend to be stable, so work at tier level for anything you want to rely on.)

Explain what is happening, and tell us what — if anything — the business should do about it.

(b) "Are our sales actually making us money?"

The game runs regular promotions; `purchase.sale_flag` marks purchases made during one. Revenue on sale days is dramatically higher than on other days, and the monetisation team wants to run sales more often.

Finance disagrees. Their argument: players have learned the promotion schedule and simply wait for it, so a sale mostly changes *when* someone buys rather than *whether* they buy.

Who is right? Should we run more sales, fewer, or the same number? Quantify your answer as best the data allows.

For either question, we are looking for a clear framing, an honest treatment of what the data can and cannot support, and a recommendation you are willing to defend. Both questions have a tempting answer that falls apart under scrutiny, so please do scrutinise. If your conclusion is "the data cannot settle this, and here is the experiment I would run instead," that is a strong answer — provided you show the work that got you there.

3. Is one acquisition channel better than another?

The UA team spends across three channels — `organic`, `paid_social` and `paid_video` — and wants to shift budget toward whichever brings the best players.

Answer whether the channels genuinely differ in player quality, and quantify how confident you are. (Aim for 60–90 minutes on this one.)

At minimum:

- Compare the channels on **at least one engagement measure and one revenue measure**. Define each precisely.
- Put a **confidence interval** on your headline difference, and state whether it is statistically significant.
- Say what you would need to conclude a difference is *real* rather than an artefact of how the data was collected.

Be careful here. At least one of the obvious comparisons is badly misleading, and at least one genuine difference is too small for this sample to resolve. "The difference is real but we cannot prove it at this sample size" and "the difference is an artefact" are both excellent answers — but they are different answers, and we want to see which one applies where.

A statistically significant result is not automatically a business-relevant one. Tell us whether the difference is big enough to move budget over.

4. Predict which players are about to churn

Predict which currently active players will stop playing in the next 14 days.

The live-ops team wants to target re-engagement offers, and offers cost money — so a model that tells them *who* to target is only useful alongside a view of how many people they would be wasting the offer on.

You will need to **define churn from the data yourself**; justify the definition you pick rather than reaching for a convention. Your write-up must cover:

- **Setup.** What exactly you predict, as of when, for which players. Be explicit about who you excluded and why.
- **Evaluation.** How you split the data and why that split is appropriate here. Which metrics you chose, and what they do and do not tell you.
- **Results.** How well it works, and where it works badly.
- **Use.** Where you would set the threshold, and how that choice depends on what an offer costs versus what a retained player is worth.

Model sophistication is not what we are grading. A well-understood logistic regression will score higher than a black-box ensemble you cannot interpret or justify.

4b. Optional — the value of a new install

If you have time and want to show more range: **predict how much revenue a new install will generate in its first 90 days, using only what is known about that player by the end of their first 7 days**. UA would use this to decide how much to bid for an install, so ranking players correctly matters more than being accurate on average.

Fair warning: this one is harder than it looks, and the difficulty is not in the modelling. Think carefully about which players you are able to label at all.

This is genuinely optional. A strong deliverable 4 with this left undone beats two rushed models, and we would rather see depth than breadth.

5. The readout

Maximum 8 slides, aimed at the Product Director. What you found, what you recommend, and what you are uncertain about. This limit is real — being selective is part of the skill.

Include at least one chart that carries real weight in the argument. We care that it makes the point clearly, not that it is elaborate.

Submitting

A single zip or repo containing:

- **Your submission as one slide deck** (.pptx , or exported from whichever tool you use). The readout (section 5 above) comes first — **8 slides maximum, and that limit is real**. Behind it, in appendix slides, put deliverables 1–4 in more depth: your numbers, the judgement calls you made and why, the data-quality issues you hit and how you handled each, dead ends, and what you would do with another week. The appendix is not length-limited, and we do read it.
- **Your working files**. The code and queries the deck is built on — whatever you actually worked in, in whichever language you are strongest in. **We have no preference**. They need to run, and we should be able to follow them.

Questions are welcome — email us. Asking a good clarifying question is not a mark against you.